

ЛАБОРАТОРНА РОБОТА №4

Тема: Створення веб-сайтів із використанням засобів React

Мета: ознайомитись із React фреймворком; навчитись створювати веб-сайти використовуючи React

Теоретичні відомості

React (іноді його називають React.js або ReactJS) - це бібліотека JavaScript для побудови швидкого та інтерактивного інтерфейсу користувача.

React часто називають фреймворком через його поведінку та можливості. Але технічно це бібліотека.

Створюючи додаток за допомогою React, ви створюєте купу незалежних, ізольованих та багаторазових компонентів. Подумайте про компонент як просту функцію, яку ви можете викликати за допомогою певного вводу, і вони надають певний результат.

І як ми можемо повторно використовувати функції, так ми також можемо повторно використовувати компоненти, об'єднувати їх і тим самим створювати складний користувацький інтерфейс.

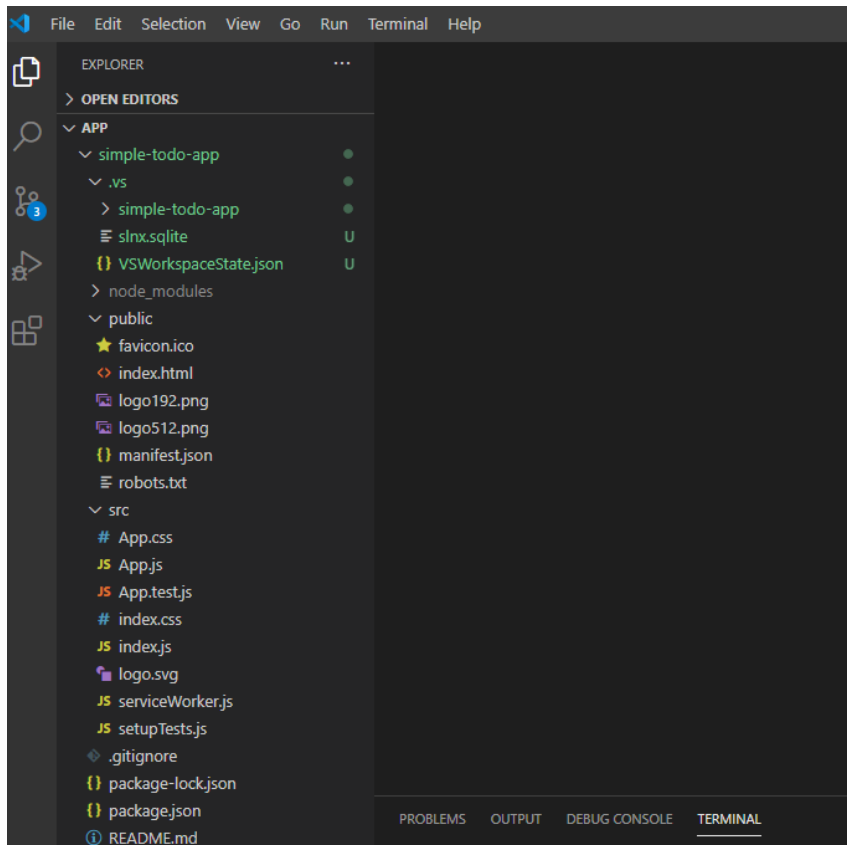
Хід роботи

1. На першому кроці нам необхідно встановити [Node.js](https://nodejs.org/):

2. Створимо новий проект за допомогою команди:

```
npx create-react-app simple-todo-app
```

3. В результаті виконання попередньої команди ми отримаємо проект із наступною структурою:



Папка `node_modules` містить усі сторонні бібліотеки, а також сам React. Він також міститиме пакети, які ви будете встановлювати через `npm` пізніше. Загальнодоступна папка містить загальнодоступний актив вашої програми, і там знаходяться ваші статичні файли.

`Index.html` у загальній папці схожий на той, який ми створили раніше. У ньому також є елемент контейнера `div`, де відображатиметься вся ваша програма.

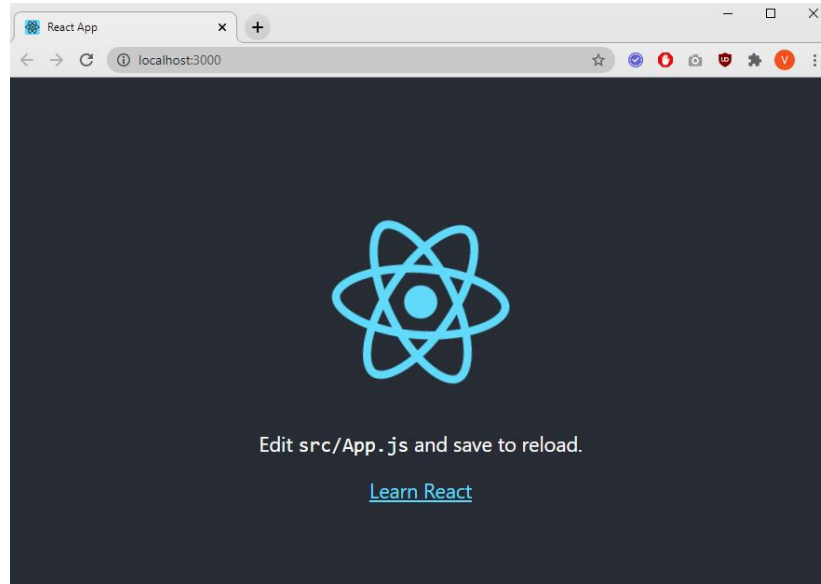
Папка `src` містить робочі файли. Одним з них є `index.js`, який послужить точкою входу в наш додаток. Не хвилюйтеся про всі файли `src`, ми напишемо все з нуля.

Нарешті, `package.json` містить інформацію про вашу програму. Він має деякі залежності бібліотек, які встановлені на даний момент, і якщо ви встановите інші пакети, вони також будуть перераховані

4. Для того щоб запускати наш веб-сайт використовуйте команду:

```
npm start
```

В результаті отримаємо:



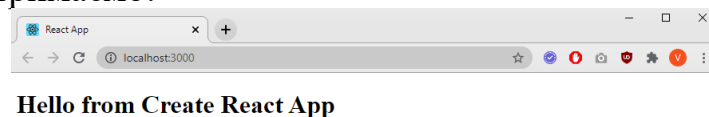
5. Наступним кроком необхідно вставити наступний код в файл `index.js` що знаходиться в `src` папці:

```
import React from "react"
import ReactDOM from "react-dom"

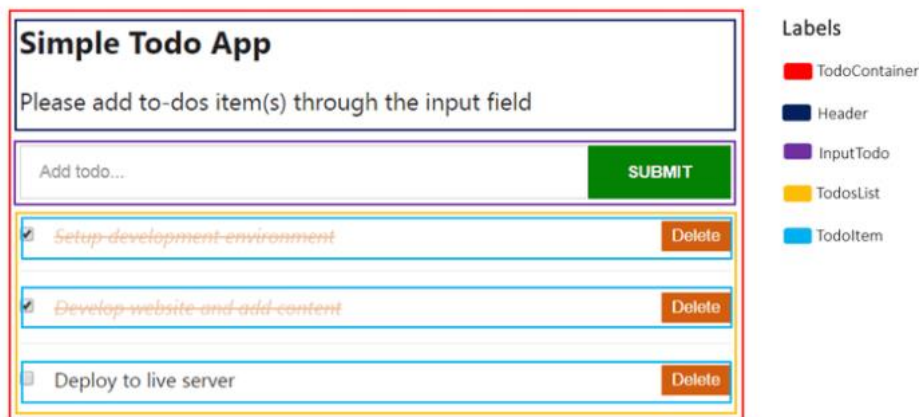
const element = <h1>Hello from Create React App</h1>

ReactDOM.render(element, document.getElementById("root"))
```

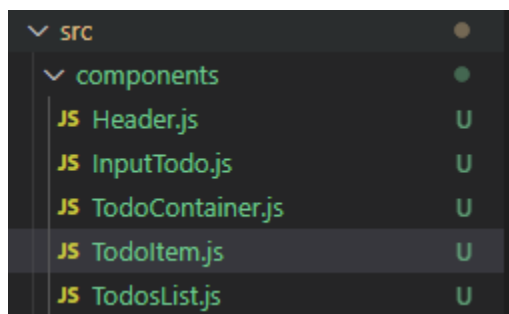
В результаті отримаємо:



6. Спочатку розкладемо наш додаток на дерево ізольованих компонентів. Там, де батьківський компонент, `TodoContainer`, містить три дочірні компоненти (`Header`, `InputTodo` та `TodosList`). Потім `TodosList` містить ще один компонент, який називається `TodoItem`:



7. Давайте створимо ці файли. Почніть із створення папки з назвою `components` всередині каталогу `src` та створіть файли цих компонентів - тобто `TodoContainer.js`, `Header.js`, `InputTodo.js`, `TodosList.js` та `TodoItem.js`:



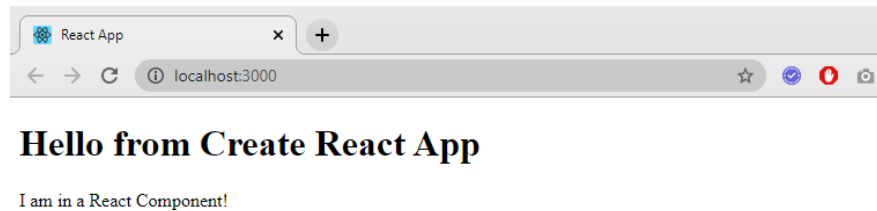
8. Додайте наступний код `TodoContainer.js`:

```
1  import React from "react"
2  class TodoContainer extends React.Component {
3    render() {
4      return (
5        <div>
6          <h1>Hello from Create React App</h1>
7          <p>I am in a React Component!</p>
8        </div>
9      )
10   }
11 }
12 export default TodoContainer
```

9. Змініть index.js файл так щоб він мав такий вигляд:

```
src > JS index.js
1   import React from "react"
2   import ReactDOM from "react-dom"
3   //component file
4   import TodoContainer from "../components/TodoContainer"
5   ReactDOM.render(<TodoContainer />, document.getElementById("root"))
```

В результаті отримаємо:



По-перше, щоб створити компонент у React, ви або пишете його як клас, або на основі функціоналу. Компонент класу створюється за допомогою синтаксису класу ES6, тоді як функціональний компонент створюється за допомогою функції запису.

У батьківському файлі нашого додатку ми визначили клас компонента React під назвою TodoContainer і розширюємо клас Component у бібліотеці React. Усередині цього класу ми маємо метод render (), де ми повертаємо JSX, який відображається на екрані.

Переконайтеся, що ви імпортували об'єкт React для роботи JSX.

Одразу ми створили батьківський компонент, ми відтворили його за допомогою користувацького тегу, подібного до HTML, <TodoContainer /> у файлі index.js. Тепер замість того, щоб відображати простий елемент JSX, ми відображаємо компонент React.

Це необхідно, щоб його екземпляр (наприклад, <TodoContainer />) у JSX не розглядався як тег DOM / HTML. Крім того, зверніть увагу на шлях до файлу

компонента, який використовується у файлі `index.js`. Обов'язково завжди вказуйте відносний шлях до цього файлу з поточного каталогу.

У нашому випадку `"/components/ToDoContainer"`. Це означає, що файл `ToDoContainer` знаходиться в папці компонентів у поточному каталозі.

За замовчуванням розширення файлу `.js`, тому вам не потрібно додавати його.

Як ми маємо на схемі програми, компонент `InputTodo` нестиме відповідальність за прийняття вводу користувача. Це означає, що нам знадобиться місце для зберігання даних (у цьому випадку сховище - це стан), отриманих через вхід. Звідти ми можемо відображати дані в інтерфейсі.

Аналогічно, компонент `TodosList` буде отримувати доступ до даних стану та відображати його елементи `todos`. Крім того, компонент `TodoItem` (який утримує прапорець та кнопку видалення) матиме доступ до даних, щоб оновити прапорець, а також видалити елементи зі стану.

Тепер для кожного компонента, який буде отримувати доступ до даних стану, вам потрібно буде оголосити об'єкт стану у файлі їх найближчого спільного батька. З цієї причини дані стану будуть розміщені в компоненті `ToDoContainer`, який є їх найближчим спільним батьком.

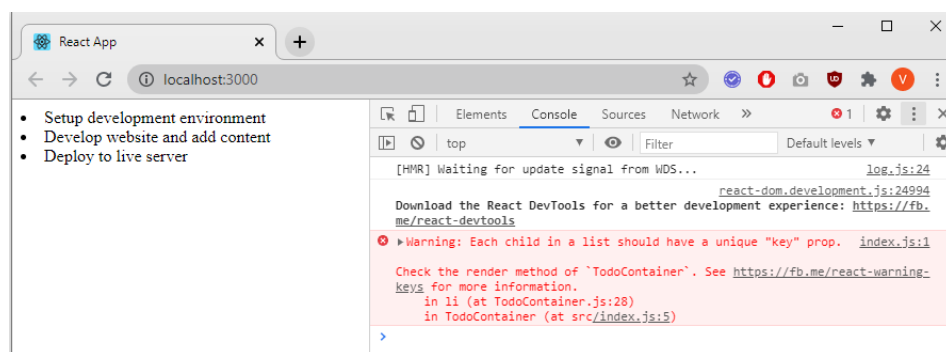
10. У випадку даних `todos` ми матимемо масив об'єктів. Тож додайте наступний код трохи вище методу `render ()` у файлі `ToDoContainer.js`:

```
JS index.js JS TodoContainer.js X JS Header.js JS InputTodo.js JS TodosList.js JS TodoItem.js
src > components > JS TodoContainer.js > TodoContainer > state
1 import React from "react"
2
3
4 class TodoContainer extends React.Component {
5   state = {
6     todos: [
7       {
8         id: 1,
9         title: "Setup development environment",
10        completed: true
11      },
12      {
13        id: 2,
14        title: "Develop website and add content",
15        completed: false
16      },
17      {
18        id: 3,
19        title: "Deploy to live server",
20        completed: false
21      }
22    ]
23  }
24 }
```

11. В цьому ж файлі змініть метод `render()`:

```
24 render() {
25   return (
26     <div>
27       {this.state.todos.map(todo => (
28         <li>{todo.title}</li>
29       ))}
30     </div>
31   );
32 }
33 }
```

В результаті отримаємо:



Після того, як ми визначили дані `todos` в об'єкті стану, ми отримали доступ до них у методі `render()`, використовуючи `this.state.todos`. Оскільки його значення - це масив об'єктів, як оголошено у стані, ми прокрутили цей масив і вивели кожен із елементів `todos`, тобто заголовок.

У React ми використовуємо метод `map()`, який є функцією вищого порядку для виконання цієї ітерації.

Якщо перевірити консоль DevTools, ви побачите попередження React. Замість ми подбаємо про це.

Ви зрозумієте, що інший компонент під назвою `TodosList` несе відповідальність за обробку списку `todos`.

Що ми хочемо зробити, це передати дані стану з `TodoContainer` в компонент `TodosList`.

12.Спочатку зайдіть у файл `TodosList.js` і створіть компонент під назвою `TodosList`:

```
src > components > JS TodosList.js > [0] default
1  import React from "react"
2
3  class TodosList extends React.Component {
4    render() {
5      return (
6        <div>
7          {this.props.todos.map(todo => (
8            <li>{todo.title}</li>
9          ))}
10       </div>
11     )
12   }
13 }
14
15 export default TodosList
```

13.Після цього відкрийте файл `TodoContainer.js` і змініть метод `render()`, щоб він виглядав так:

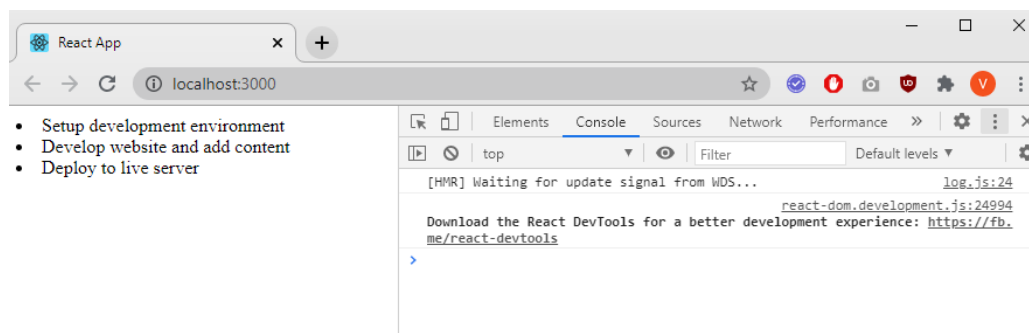
```
2  import TodosList from "../TodosList";
3
4
5  class TodoContainer extends React.Component {
6    state = { ...
24   };
25   render() {
26     return (
27       <div>
28         <TodosList todos={this.state.todos} />
29       </div>
30     );
31   }
32 }
33 export default TodoContainer
```


Дані todos, що надходять із стану компонента TodoContainer, передаються як пропси, використовуючи `todos = {this.state.todos}`. Потім ми отримали доступ до нього через `this.props.todos` із компонента TodosList.

14. Тепер необхідно виправити помилки в консолі. Кожного разу, коли ви переглядаєте щось, створюється список. React хоче, щоб кожна дитина у списку мала унікальний ключовий реквізит. Це допомагає React визначити, які елементи були змінені, додані чи видалені. Щоб додати цей унікальний ключовий опис, ми скористаємося ідентифікатором, який ми надали у стані TodoContainer. Ми можемо отримати доступ до цих ідентифікаторів так само, як ми отримали доступ до заголовка. Тож продовжуйте та оновлюйте елемент `<li>` у компоненті TodosList, щоб ви мали:

```
src > components > JS TodosList.js > TodosList > render > props.todos.map() callback
1  import React from "react"
2
3  class TodosList extends React.Component {
4    render() {
5      return (
6        <div>
7          {this.props.todos.map(todo => (
8            <li key={todo.id}>{todo.title}</li>
9          ))}
10         </div>
11       )
12     }
13   }
14
15   export default TodosList
```

В результаті отримаємо:



15.Знову ж таки, якщо ви повернетесь до схеми програми, ви зрозумієте, що інший компонент під назвою `TodoItem` несе відповідальність за обробку кожного елемента завдання. Ми робили щось подібне раніше. Відкрийте файл `TodoItem.js` і створіть компонент під назвою `TodoItem`:

```
src > components > JS TodoItem.js > [⌘] default
1  import React from "react"
2
3  class TodoItem extends React.Component {
4    render() {
5      return <li>{this.props.todo.title}</li>
6    }
7  }
8
9  export default TodoItem
```

16. Далі імпортуйте компонент у файл `TodosList.js`, та замість `<li>` в методі `map` помістіть наступне:

```
src > components > JS TodosList.js > TodosList > render > props.todos.map() callback
1  import React from "react"
2  import TodoItem from "../TodoItem";
3
4  class TodosList extends React.Component {
5    render() {
6      return (
7        <div>
8          {this.props.todos.map(todo => (
9            <TodoItem key={todo.id} todo={todo} />
10          ))}
11        </div>
12      )
13    }
14  }
15
16  export default TodosList
```

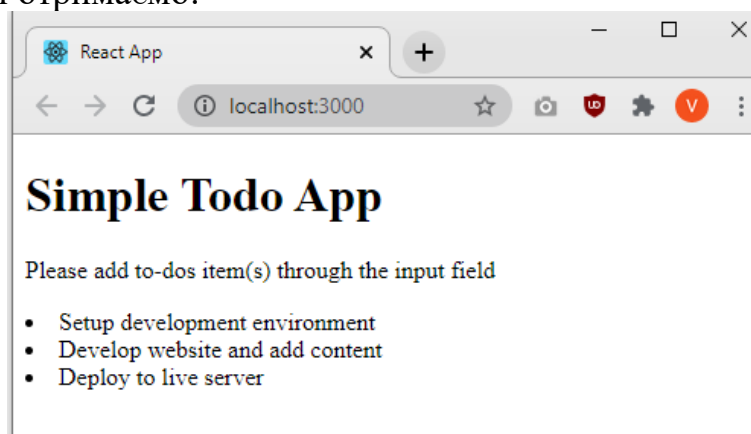
17. Додайте наступний код у файл `Header.js`

```
src > components > JS Header.js > [⌕] default
1  import React from "react"
2
3  const Header = () => {
4    return (
5      <header>
6        <h1>Simple Todo App</h1>
7        <p>Please add to-dos item(s) through the input field</p>
8      </header>
9    )
10 }
11
12 export default Header
```

18.Імпоруйте Header.js в TodoContainer.js файл та додайте об'єкт Header в render метод:

```
3  import Header from "../Header"
4
5
6  class TodoContainer extends React.Component {
7    > state = { ...
25    };
26    render() {
27      return (
28        <div>
29          <Header />
30          <TodosList todos={this.state.todos} />
31        </div>
32      );
33    }
34  }
35  export default TodoContainer
```

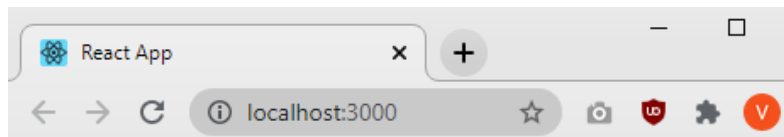
В результаті отримаємо:



19.В TodoItem.js додайте input перед {this.props.todo.title} в елементі li

```
1 import React from "react"
2
3 class TodoItem extends React.Component {
4   render() {
5     return <li><input type="checkbox" />{this.props.todo.title}</li>
6   }
7 }
8
9 export default TodoItem
```

В результаті отримаємо:



Simple Todo App

Please add to-dos item(s) through the input field

- ☒ Setup development environment
- ☐ Develop website and add content
- ☐ Deploy to live server

20.Всередині файлу TodoItem.js та додайте checked до input, а потім призначте {this.props.todo.completed}.

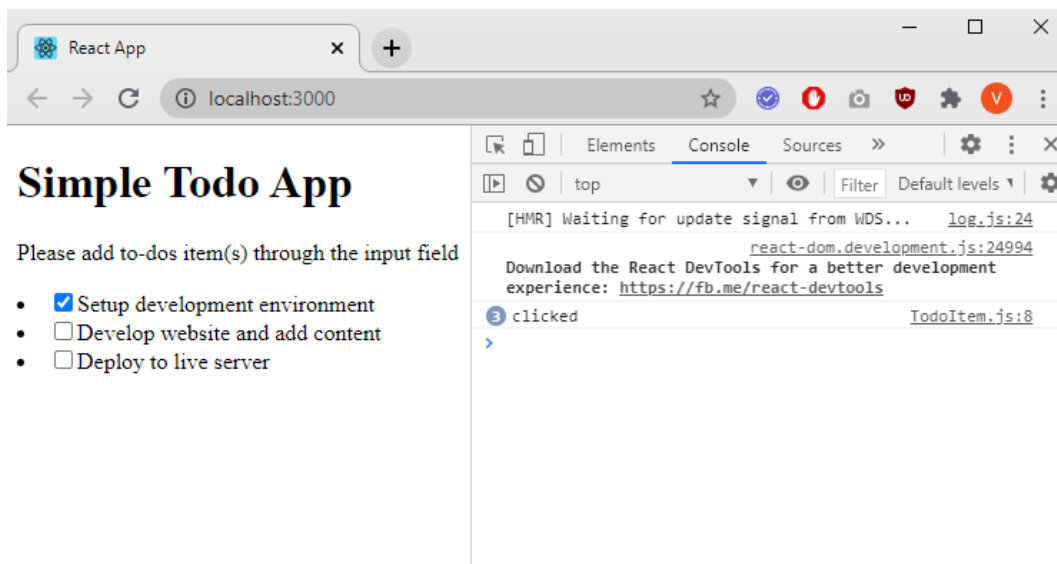
```
src > components > JS TodoItem.js > TodoItem > render
1 import React from "react"
2
3 class TodoItem extends React.Component {
4   render() {
5     return <li><input type="checkbox" checked={this.props.todo.completed} />{this.props.todo.title}</li>
6   }
7 }
8
9 export default TodoItem
```

21.Тепер, якщо ви спробуєте перемкнути будь-який прапорець, нічого не трапиться. Це тому, що кожному з перевірених атрибутів присвоюється значення, рівне поточному значенню стану. Пам'ятайте, що виконується лише перше завдання. Нам потрібен спосіб змінити стан кожного разу, коли користувачі натискають прапорці. React вже дає нам підказку через вкладку Консоль браузера DevTools. Якщо його відкрити, ви побачите попередження

як результат доданого перевіреного атрибута. React каже нам додати обробник onChange, щоб відстежувати будь-які зміни в полі. В іншому випадку, він не знав би, чи введено поле введення, чи ні. Тож давайте оновимо вхідний тег у файлі TodoItem.js, щоб включити обробник.

```
src > components > JS TodoItem.js > [O] default
1  import React from "react"
2
3  class TodoItem extends React.Component {
4    render() {
5      return <li>
6        <input
7          type="checkbox"
8          checked={this.props.todo.completed}
9          onChange={() => console.log("clicked")} />
10       {this.props.todo.title}
11     </li>
12   }
13 }
14
15 export default TodoItem
```

В результаті отримаємо:



У нашому додатку, батьківським компонентом, TodoContainer є той, що містить дані стану. Отже, цей компонент - ЄДИНИЙ, який може його змінити. Це означає, що компонент TodoItem, який обробляє прапорці, не може змінити дані стану в батьківському компоненті, TodoContainer.

Нам потрібно знайти спосіб отримати доступ до даних стану з `TodoItem` і перемкнути завершене значення на `true` або `false` у компоненті `TodoContainer`.

Для цього нам потрібно буде підняти подію з `TodoItem` на рівень до `TodosList`, а потім у компонент `TodoContainer`. Іншими словами, нам потрібно піднятися сходами.



Компонент `TodoItem` буде викликати подію, тоді як батьківський компонент, `TodoContainer` буде обробляти подію. І ми робимо це через `props`.

22.Спочатку ми дозволимо зв'язок між цими компонентами. Починаючи з батьківського компонента, `TodoContainer`, додайте метод обробника, `handleChange` трохи вище методу `render ()`:

```
src > components > JS TodoContainer.js > TodoContainer
1  import React from "react"
2  import TodosList from "../TodosList";
3  import Header from "../Header"
4
5
6  class TodoContainer extends React.Component {
7    state = { ...
25  };
26
27    handleChange = () => {
28      console.log("clicked");
29    };
30
31    render() {
32      return (
33        <div>
34          <Header />
35          <TodosList todos={this.state.todos} />
36        </div>
37      );
38    }
39  }
40  export default TodoContainer
```

23. Почніть з передачі цього методу компоненту TodosList через props. Тож оновіть `<TodosList />`, щоб ви мали:

```
src > components > JS TodoContainer.js > TodoContainer > render
1  import React from "react"
2  import TodosList from "../TodosList";
3  import Header from "../Header"
4
5
6  class TodoContainer extends React.Component {
7    state = { ...
25  };
26
27    handleChange = () => {
28      console.log("clicked");
29    };
30
31    render() {
32      return (
33        <div>
34          <Header />
35          <TodosList todos={this.state.todos} handleChangeProps={this.handleChange} />
36        </div>
37      );
38    }
39  }
40  export default TodoContainer
```

24. Тепер у вас є метод `handleChange()`, призначений `handleChangeProps`. До його даних можна отримати доступ через props у компоненті TodosList. Звідти ми можемо передати його компоненту TodoItem. Давайте оновимо екземпляр `<TodoItem />` у файлі TodosList.js, щоб ви мали:

```
src > components > JS TodosList.js > TodosList > render > props.todos.map() callback
1  import React from "react"
2  import TodoItem from "../TodoItem";
3
4  class TodosList extends React.Component {
5    render() {
6      return (
7        <div>
8          {this.props.todos.map(todo => (
9            <TodoItem
10             key={todo.id}
11             todo={todo}
12             handleChangeProps={this.props.handleChangeProps}
13           )}
14        </div>
15      )
16    }
17  }
18
19
20  export default TodosList
```

25. На даний момент доступ до даних `handleChange()` можна отримати з компонента `TodoItem`. Тож оновіть обробник `onChange` у компоненті `TodoItem`, щоб у вас було:

```
src > components > JS TodoItem.js > TodoItem > render
1  import React from "react"
2
3  class TodoItem extends React.Component {
4    render() {
5      return <li>
6        <input
7          type="checkbox"
8          checked={this.props.todo.completed}
9          onChange={() => this.props.handleChangeProps()} />
10       {this.props.todo.title}
11     </li>
12   }
13 }
14
15 export default TodoItem
```

Цього разу переконайтеся, що у вас є дужки, `()`, прикріплені до `handleChangeProps`.

Збережіть усі файли. Тепер, якщо ви клацнете будь-який прапорець, подія `onChange` спрацює і викличе метод `handleChange()` у батьківському компоненті, `TodoContainer`.

Наразі ми реєструємо лише текст у консолі.

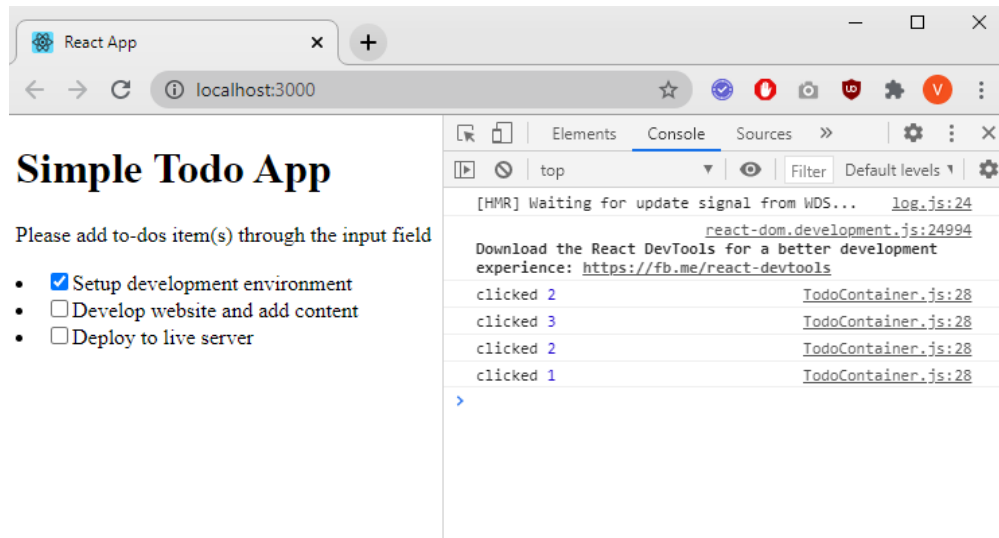
26. Оновіть обробник `onChange` у компоненті `TodoItem`, щоб включити ідентифікатор:

```
src > components > JS TodoItem.js > TodoItem > render
1  import React from "react"
2
3  class TodoItem extends React.Component {
4    render() {
5      return <li>
6        <input
7          type="checkbox"
8          checked={this.props.todo.completed}
9          onChange={() => this.props.handleChangeProps(this.props.todo.id)} />
10       {this.props.todo.title}
11     </li>
12   }
13 }
14
15 export default TodoItem
```

27. Увійдіть до компонента `TodoContainer` і оновіть метод `handleChange`:

```
src > components > JS TodoContainer.js > TodoContainer
1  import React from "react"
2  import TodosList from "../TodosList";
3  import Header from "../Header"
4
5
6  class TodoContainer extends React.Component {
7    state = { ...
25   };
26
27   handleChange = (id) => {
28     console.log("clicked", id);
29   };
30
31   render() {
32     return (
33       <div>
34         <Header />
35         <TodosList todos={this.state.todos} handleChangeProps={this.handleChange} />
36       </div>
37     );
38   }
39 }
40 export default TodoContainer
```

В результаті отримаємо:



Наша мета - змінити стан прапорця щоразу, коли на нього клацають. У React ми не змінюємо стан безпосередньо. Натомість ми оновлюємо метод, який ми успадкували, розширивши `React.Component`. Цей метод називається `setState()`. Це повідомляє React, що ми оновлюємо стан. React з'ясовує, яка частина стану змінена, а потім оновить ТІЛЬКИ ту частину в реальному DOM. Все, що нам потрібно зробити в методі `handleChange`, це перевірити, чи відповідає ідентифікатор клікнутого прапорця будь-якому з ідентифікаторів елементів `todos`. Якщо це станеться, ми перетворимо заповнене значення з `true` на `false` і навпаки.

28. Тепер зайдіть у компонент `TodoContainer` і оновіть метод `handleChange`, щоб він виглядав так:

```
27  handleChange = id => {
28    this.setState({
29      todos: this.state.todos.map(todo => {
30        if (todo.id === id) {
31          todo.completed = !todo.completed;
32        }
33        return todo;
34      })
35    });
36  };
37
```

29.Видалення елементів із списку буде подібним до того, як ми обробляли прапорці введення. Тут також елементи todos знаходяться в TodoContainer, тоді як кнопка видалення - у компоненті TodoItem. Це означає, що ми будемо піднімати подію з компонента TodoItem і рухатись вгору, поки не дійдемо до компонента TodoContainer, де буде оброблятися подія. Почніть з додавання кнопки видалення в компонент TodoItem:

```
src > components > JS TodoItem.js > TodoItem > render
1  import React from "react"
2
3  class TodoItem extends React.Component {
4    render() {
5      return <li>
6        <input
7          type="checkbox"
8          checked={this.props.todo.completed}
9          onChange={() => this.props.handleChangeProps(this.props.todo.id)} />
10       <button>Delete</button>
11       {this.props.todo.title}
12     </li>
13   }
14 }
15
16 export default TodoItem
```

30.Всередині компонента TodoContainer і додайте метод delTodo над render():

```
37
38   delTodo = id => {
39     console.log("deleted", id);
40   };
41
```

31.В цьому ж компоненті оновіть <TodosList />:

```
41
42   render() {
43     return (
44       <div>
45         <Header />
46         <TodosList todos={this.state.todos} handleChangeProps={this.handleChange} deleteTodoProps={this.delTodo} />
47       </div>
48     );
49   }
50 }
51 export default TodoContainer
```

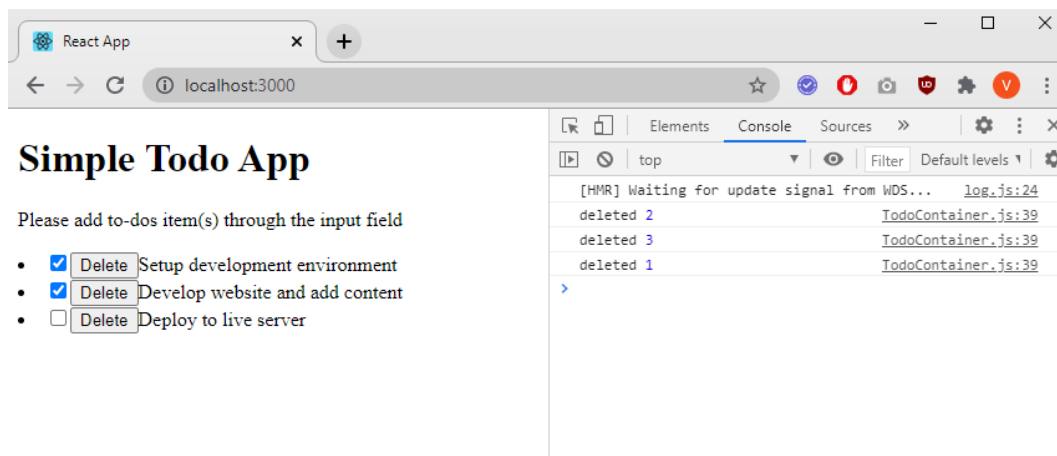
32.Оновіть <TodoItem />:

```
src > components > JS TodosList.js > TodosList > render > props.todos.map() callback
1 import React from "react"
2 import TodoItem from "../TodoItem";
3
4 class TodosList extends React.Component {
5   render() {
6     return (
7       <div>
8         {this.props.todos.map(todo => (
9           <TodoItem
10             key={todo.id}
11             todo={todo}
12             handleChangeProps={this.props.handleChangeProps}
13             deleteTodoProps={this.props.deleteTodoProps}
14           />
15         ))}
16       </div>
17     )
18   }
19 }
20
21 export default TodosList
```

33. Нарешті, повернімось до компонента TodoItem. Оновіть елемент button, щоб включити обробник події onClick, який ініціюватиме метод delTodo у батьківському компоненті:

```
src > components > JS TodoItem.js > TodoItem > render
1 import React from "react"
2
3 class TodoItem extends React.Component {
4   render() {
5     return <li>
6       <input
7         type="checkbox"
8         checked={this.props.todo.completed}
9         onChange={() => this.props.handleChangeProps(this.props.todo.id)} />
10      <button onClick={() => this.props.deleteTodoProps(this.props.todo.id)}>Delete</button>
11      {this.props.todo.title}
12    </li>
13   }
14 }
15
16 export default TodoItem
```

В результаті отримаємо:



34.Видалені елементи зі списку. Ми робимо це за допомогою методу `filter()`. Цей метод також є функцією вищого порядку, як і метод `map()`. Він повертає новий масив, застосовуючи умову до кожного елементу масиву. У цьому випадку ми хочемо повернути лише елементи `todos`, які не відповідають ідентифікатору, який буде передано - тобто ідентифікатору, що натиснуто. Будь-який ідентифікатор, який відповідає, буде видалено. Тепер оновіть метод `delTodo`, щоб ви мали:

```

38      delTodo = id => {
39        this.setState({
40          todos: [
41            ...this.state.todos.filter(todo => {
42              return todo.id !== id;
43            })
44          ]
45        });
46      };
47

```

35.Додавання поля введення тексту та кнопки submit. У React усі різні типи полів введення дотримуються однакового підходу. Ми побачили, як працює тип поля введення. Використовуючи той самий шаблон, ми додамо поле введення тексту, яке дозволить користувачам додавати елементи списку до списку. Додамо наступний код в `InputTodo.js`:

```

src > components > JS InputTodo.js > [⌕] default
1  import React, { Component } from "react"
2
3  class InputTodo extends Component {
4    render() {
5      return (
6        <form>
7          <input type="text" placeholder="Add Todo..." />
8          <input type="submit" value="Submit" />
9        </form>
10     )
11   }
12 }
13 export default InputTodo

```

36. Додамо InputToDo.js компонент в InputContainer.js та додамо `<InputToDo />` в середину методу `render()`:

```
src > components > JS TodoContainer.js > TodoContainer > render
1  import React from "react"
2  import TodosList from "../TodosList";
3  import Header from "../Header"
4  import InputToDo from "../InputToDo"
5
6
7  class TodoContainer extends React.Component {
8  >   state = { ...
26   };
27
28 >   handleChange = id => { ...
37   };
38
39 >   delTodo = id => { ...
47   };
48
49   render() {
50     return (
51       <div>
52         <Header />
53         <InputToDo />
54         <TodosList todos={this.state.todos} handleChangeProps={this.handleChange} deleteTodoProps={this.delTodo}/>
55       </div>
56     );
57   }
58 }
59 export default TodoContainer
```

37. Додайте наступний код вище методу `render` в `InputToDo.js`:

```
4   state = {
5     |   title: ""
6   };
7
```

38. Додайте `value` для `input` в цьому ж компоненті:

```
11  <input type="text" placeholder="Add todo..." value={this.state.title} />
```

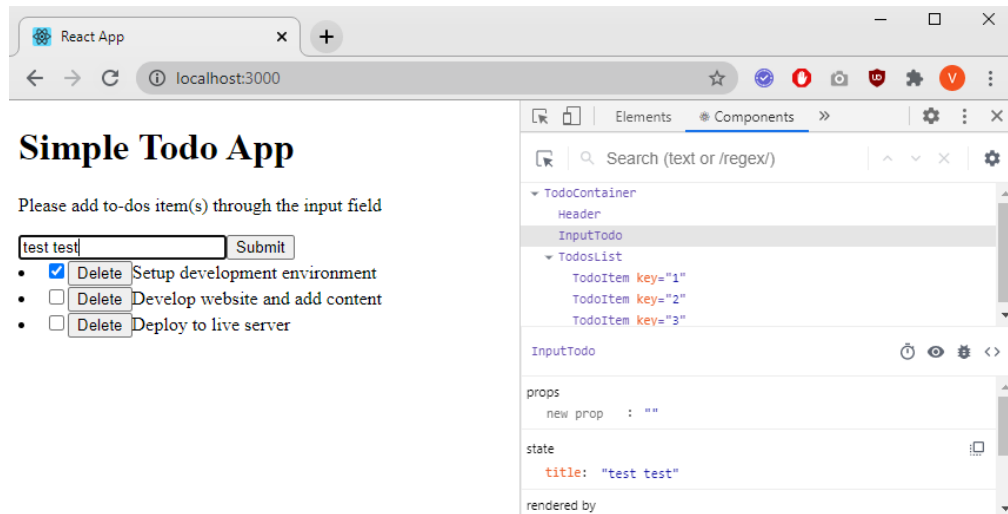
39. Додайте `onChange()` для цього ж `input`:

```
11  <input type="text" placeholder="Add todo..." value={this.state.title} onChange={this.onChange} />
```

40. Додайте наступний код вище методу `render` в цьому ж компоненті:

```
7  >   onChange = e => {
8  >     |   this.setState({
9  >     |     |   title: e.target.value
10 >     |   });
11 >     |   };
12
13
```

В результаті отримаємо:



41.Змінемо метод onChange():

```
7     onChange = e => {  
8       this.setState({  
9         [e.target.name]: e.target.value  
10      });  
11    };
```

42.Після усіх змін файл InputTodo.js має мати наступний вигляд:

```
src > components > JS InputTodo.js > [🔍] default  
1  import React, { Component } from "react"  
2  
3  class InputTodo extends Component {  
4    state = {  
5      title: "",  
6    }  
7    onChange = e => {  
8      this.setState({  
9        [e.target.name]: e.target.value,  
10      })  
11    }  
12    render() {  
13      return (  
14        <form>  
15          <input  
16            type="text"  
17            placeholder="Add todo..."  
18            value={this.state.title}  
19            name="title"  
20            onChange={this.onChange}  
21          />  
22          <input type="submit" value="Submit" />  
23        </form>  
24      )  
25    }  
26  }  
27  export default InputTodo
```

43.Оновлення списку завдань На даний момент сторінка перезавантажиться, якщо ви спробуєте надіслати пункт завдання та оновити дані про стан. Нам потрібно з цим впоратися. Для подання запитів, ми використаємо обробник події onSubmit для елемента форми. Давайте швидко зробимо це. Оновіть тег <form> у компоненті InputTodo, щоб включити обробник onSubmit:

```
14   <form onSubmit={this.handleSubmit}>
15     <input
```

44.Добавимо наступний код вище методу render():

```
12   handleSubmit = e => {
13     e.preventDefault();
14     console.log(this.state.title);
15   };
16
```

45.Добавте наступний код вище методу render в компоненті TodoContainer:

```
49   addTodoItem = title => {
50     console.log(title);
51   };
52
```

46.Далі передайте цей метод класу компоненту InputTodo, оновивши <InputTodo />:

```
57   <InputTodo addTodoProps={this.addTodoItem} />
```

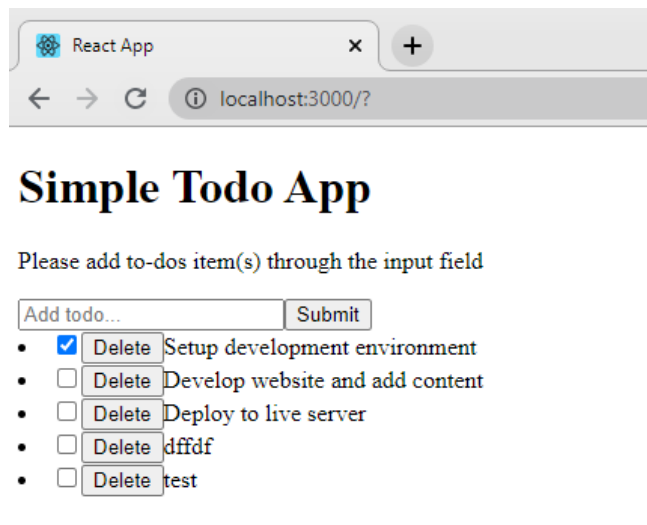
47.Тепер до методу addTodoItem () можна отримати доступ через props у компоненті InputTodo. Тож оновіть метод handleSubmit у компоненті InputTodo:

```
2   handleSubmit = e => {
3     e.preventDefault();
4     this.props.addTodoProps(this.state.title);
5     this.setState({
6       title: ""
7     });
8   };
9
```

48.Нарешті, ми можемо оновити стан. Поверніться до компонента TodoContainer, оновіть метод addTodoItem (), щоб у вас було:


```
9  ✓      addToItem = title => {  
10  ✓      const newTodo = {  
11  ✓        id: 4,  
12  ✓        title: title,  
13  ✓        completed: false  
14  ✓      };  
15  ✓      this.setState({  
16  ✓        todos: [...this.state.todos, newTodo]  
17  ✓      });  
18  ✓    };
```

В результаті отримаємо:



49.Створення випадкових ідентифікаторів для елементів списку завдань. Якщо ви спробуєте надіслати більше одного завдання, React викличе попередження консолі, яке повідомляє, що ідентифікатори / ключі мають бути унікальними. Це відбувається тому, що ми присвоюємо один ідентифікатор (ідентифікатор 4) кожному новому елементу, отриманому через поле введення. Це не ідеально. Довідка id React визначає, який елемент додано або видалено. Щоб виправити це попередження, ми встановимо щось, що генеруватиме випадкові ідентифікатори, які ми можемо використовувати. Це називається UUID (Універсальний унікальний ідентифікатор). Щоб встановити його, зупиніть сервер за допомогою CTRL + C. З вашого терміналу запустіть:

```
npm i uuid
```

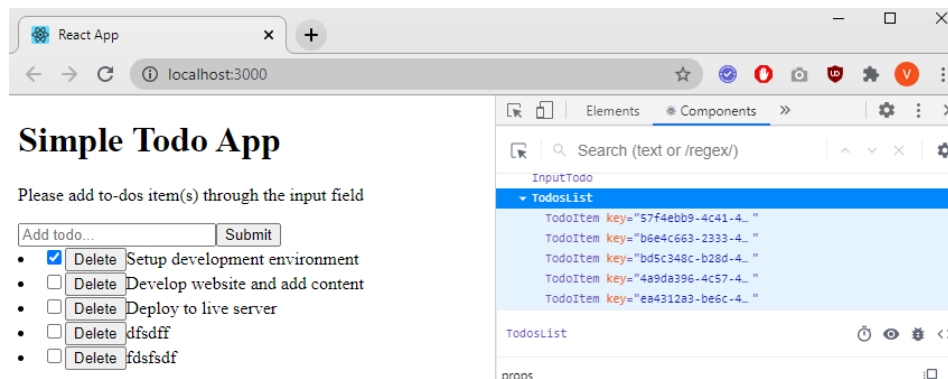
50.Щоб використовувати ці ідентифікатори у своєму додатку, вам потрібно імпортувати UUID у файлі TodoContainer.js. Отже, додайте цей рядок під списком операторів імпорту:

```
5 import { v4 as uuidv4 } from "uuid";
```

51.Потім замініть будь-яке жорстко закодоване значення ідентифікатора на uuidv4 ():

```
9 state = {
10   todos: [
11     {
12       id: uuidv4(),
13       title: "Setup development environment",
14       completed: true
15     },
16     {
17       id: uuidv4(),
18       title: "Develop website and add content",
19       completed: false
20     },
21     {
22       id: uuidv4(),
23       title: "Deploy to live server",
24       completed: false
25     }
26   ]
}
```

В результаті отримаємо:



52.Добавимо CSS в наш проект. Для цього в папці src створимо новий файл App.css і вставимо наступний код:

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
body {
```

```
font-family: "Segoe UI", Arial, sans-serif;
line-height: 1.4;
color: #1f1f1f;
}
```

53. Далі імпортуємо цей файл в `index.js`:

```
import "../App.css"
```

54.3 миємо div тег в TodoContainer на наступне:

```
<div className="container">
  <Header />
  <InputTodo addTodoProps={this.addTodoItem} />
  <TodosList
    todos={this.state.todos}
    handleChangeProps={this.handleChange}
    deleteTodoProps={this.delTodo}
  />
</div>
```

55.3мінемо form та input теги в InputTodo.js:

```
<form onSubmit={this.handleSubmit} className="form-container">
  <input
    type="text"
    className="input-text"
    placeholder="Add todo..."
    value={this.state.title}
    name="title"
    onChange={this.onChange}
  />
  <input type="submit" className="input-submit" value="Submit"
/>
</form>
```

56.3 миємо теж li в TodoItem.js:

```
<li className="todo-item">
  <input
    type="checkbox"
    checked={this.props.todo.completed}
    onChange={() =>
      this.props.handleChangeProps(this.props.todo.id) }
  />
  <button onClick={() =>
    this.props.deleteTodoProps(this.props.todo.id) }>
    Delete
  </button>
```

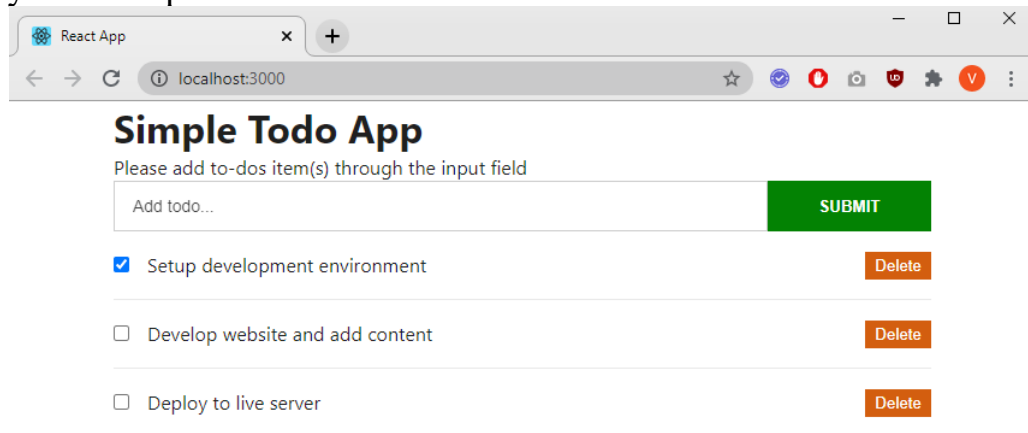
```
    {this.props.todo.title}  
</li>
```

57.І нарешті додамо наступний код в App.css:

```
.container {  
  max-width: 700px;  
  margin: 0 auto;  
  padding: 0 10px;  
}  
.form-container {  
  display: flex;  
  width: 100%;  
}  
.input-text {  
  flex: 8;  
  font-size: 14px;  
  padding: 12px 15px;  
  border: 1px solid #cecece;  
  outline: none;  
  font-weight: 400;  
  max-width: 80%;  
}  
.input-text::placeholder {  
  color: rgb(17, 17, 17);  
  opacity: 0.8;  
}  
.input-submit {  
  flex: 2;  
  border: none;  
  background: #038203;  
  color: #fff;  
  text-transform: uppercase;  
  cursor: pointer;  
  font-weight: 600;  
  width: 20%;  
  outline: none;  
}  
.todo-item {  
  list-style-type: none;  
  padding: 17px 0px;  
  border-bottom: 1px solid #eaeaea;  
}  
.todo-item input {  
  margin-right: 15px;  
}  
.todo-item button {
```

```
background: #d35e0f;
color: #fff;
border: 1px solid #d35e0f;
padding: 3px 7px;
cursor: pointer;
float: right;
outline: none;
}
```

В результаті отримаємо:



58. Якщо ви пам'ятаєте, ми використовуємо вбудований стиль у документі HTML, передаючи рядок усіх стилів атрибуту `style`. Але з React JSX ми призначимо атрибут об'єкт JavaScript. Зайдіть у файл `Header.js` та оновіть елементи заголовка та абзацу, включивши атрибут `style`:

```
return (
  <header>
    <h1 style={{ fontSize: "25px", marginBottom: "15px" }}>Simple Todo App</h1>
    <p style={{ fontSize: "19px" }}>
      Please add to-dos item(s) through the input field
    </p>
  </header>
)
```

59. В цьому ж файлі вище `return` додайте наступне:

```
const headerStyle = {
  padding: "20px 0",
  lineHeight: "2em",
}
```

60. Змініть `<header>` на:

```
<header style={headerStyle}>
```

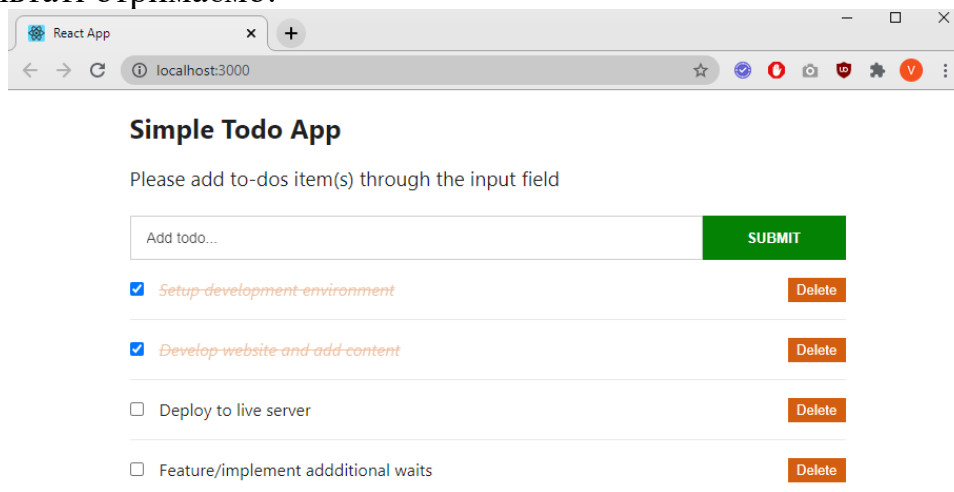
61. У компонент `TodoItem` додайте наступні стилі в метод `render ()`, але над оператором `return`:

```
const completedStyle = {
  fontStyle: "italic",
  color: "#d35e0f",
  opacity: 0.4,
  textDecoration: "line-through",
}
```

62. Існуючий `return` замініть на:

```
return (
  <li className="todo-item">
    <input
      type="checkbox"
      checked={this.props.todo.completed}
      onChange={ () }
      =>
    this.props.handleChangeProps(this.props.todo.id) }
    />
    <button
      onClick={ () }
      =>
    this.props.deleteTodoProps(this.props.todo.id) }>
      Delete
    </button>
    <span style={this.props.todo.completed ? completedStyle :
    null}>
      {this.props.todo.title}
    </span>
  </li>
)
```

В результаті отримаємо:



Індивідуальне завдання:

Реалізувати програму, яка вміє показувати такі сторінки:

- / - головна
- / Login - сторінка введення логіна і пароля
- / News - сторінка з новинами (будь-яка однотипна інформація)
- / Profile - сторінка з довільним текстом, недоступна без авторизації

На сайті, в заголовку або нижньому колонтитулі реалізувати посилання:

- На головну (/)
- Новини (/ news)
- Профіль (/ profile)

Якщо користувач клацає на сторінку "профілю" і він не "авторизований" то перекидати на сторінку / login

Форма входу (/ login) приймає "фейковий" дані:

username: Admin

password: 12345

Якщо введені інші дані, то показується повідомлення:

Ім'я користувача або пароль введені не вірно

Якщо введені коректні дані, то перекидати на сторінку / profile

Інформацію про авторизацію користувача можна зберігати в localStorage, простим параметром true / false. Базу даних реалізовувати не потрібно.

Контрольні запитання:

1. Що таке React?
2. Що відбувається, коли ви викликаєте setState?
3. Яка різниця між Елементом і Компонентом в React?
4. Що робить і чому важливий shouldComponentUpdate?
5. У чому різниця між createElement і cloneElement?